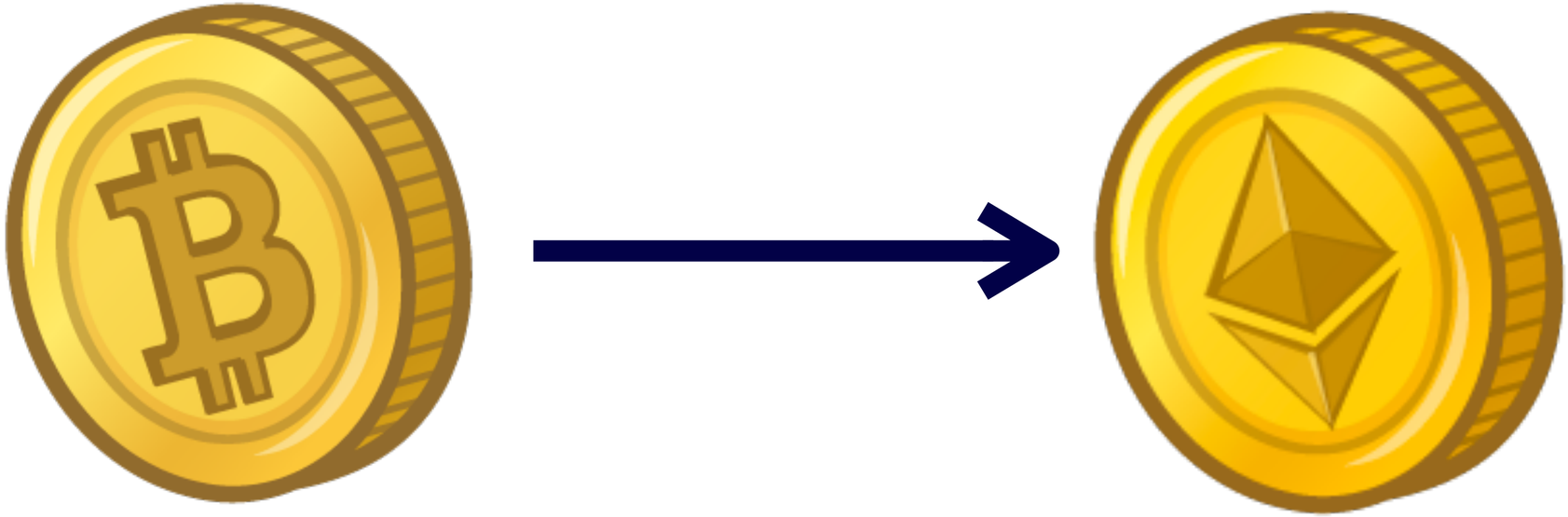
Ethereum (History)

The next generation



1st generation

2nd generation

—Ethereum

- In 2013 Vitalik Buterin published the whitepaper „ Ethereum: A Next Generation Smart Contract & Decentralized Application Platform “

"I happily played World of Warcraft during 2007- 2010, but one day Blizzard removed the damage component from my beloved warlock's Siphon Life spell. I cried myself to sleep, and on that day I realized what horrors centralized services can bring."

Vitalik Buterin

—Ethereum

- In 2013 Vitalik Buterin published the whitepaper „ Ethereum: A Next Generation Smart Contract & Decentralized Application Platform “

REAL QUOTE BUT MAYBE NOT 100% SERIOUS

"I happily played World of Warcraft during 2007- 2010, but one day Blizzard removed the damage component from my beloved warlock's Siphon Life spell. I cried myself to sleep, and on that day I realized what horrors centralized services can bring."

Vitalik Buterin

—Ethereum

- In 2013 Vitalik Buterin published the whitepaper „ Ethereum: A Next Generation Smart Contract & Decentralized Application Platform “
- The network started its operation in 2015



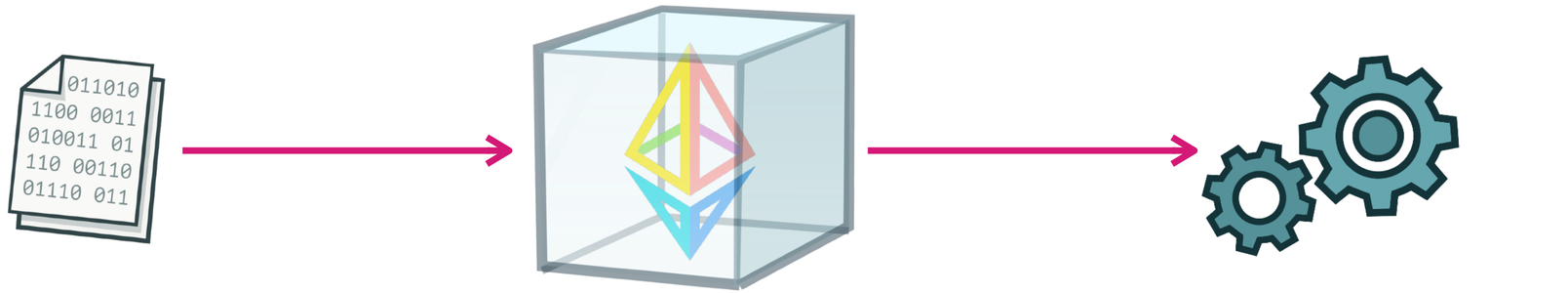
https://www.golem.de/news/kryptowaehrung-ethereum-entstand-wegen-eines-hexenmeister-nerfs-in-wow-2110-160127.html

—Ethereum

- Decentralized, open-source blockchain system featuring smart contract functionality.
- Foundational platform in the world of cryptocurrencies and blockchain technologies, playing a significant role in the development of decentralized applications (dApps).
- **Ether (ETH):** The native cryptocurrency of the Ethereum platform, used to facilitate transactions and computational services within the network.

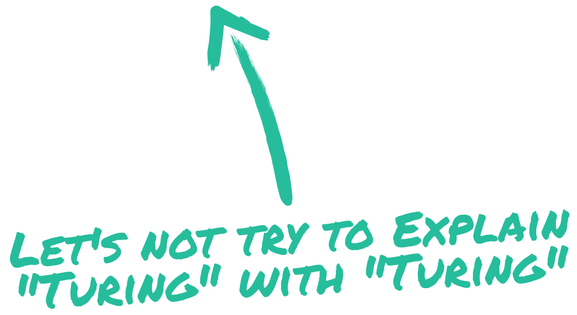
—Ethereum

- The big difference to Bitcoin is the Ethereum Virtual Machine (EVM)
- The EVM can be used to execute code (smart contracts) on the network



Turing Complete

- The EVM is Turing complete
- A Turing complete system can simulate any Turing machine



Alan Turing



Picture by wikipedia.org

- British mathematician, cryptanalyst, and computer scientist.
- Widely considered to be the father of theoretical computer science and artificial intelligence.

—Alan Turing

- **Breaking the Enigma Code:** During World War II, Turing played a pivotal role at Britain's codebreaking centre. He was instrumental in breaking the Enigma code used by Nazi Germany, significantly contributing to the Allied war effort.



Picture by wikipedia.org

—Alan Turing

- **Turing Machine:** 1936 he conceptualized the Turing machine, a theoretical device that manipulates symbols on a strip of tape according to a set of rules.
- This concept laid the groundwork for the modern theory of computation and algorithms.

Despite the model's simplicity, it is capable of implementing any computer algorithm

... if you have endless time and memory

—Alan Turing

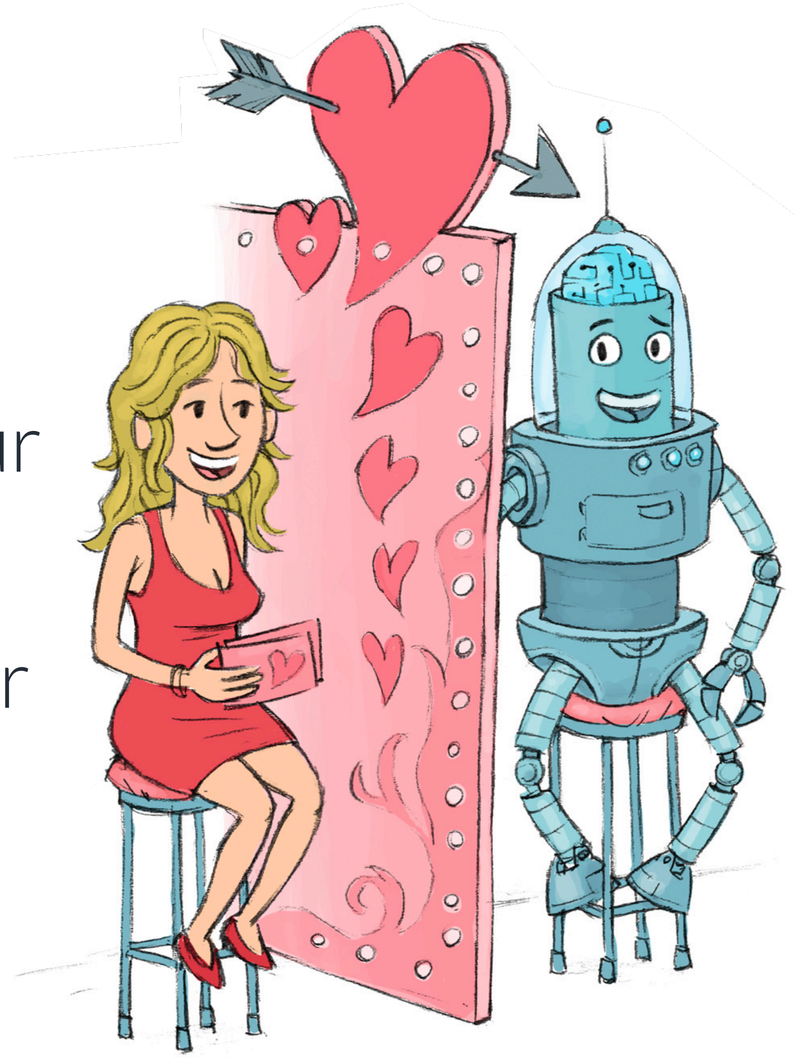
- **Turing completeness:** the ability for a computational model to simulate a Turing machine.
- A programming language that is Turing complete is theoretically capable of expressing all tasks accomplishable by computers



THE MODERN PROGRAMMING LANGUAGES WE USE TODAY ARE ALL TURIN COMPLETE

—Alan Turing

- **Turing Test:** Originally called the imitation game by Alan Turing in 1950
- Test a machine to exhibit intelligent behaviour equivalent to a human.
- If a human does not notice that a chat partner is a machine the test is passed



Ethereum

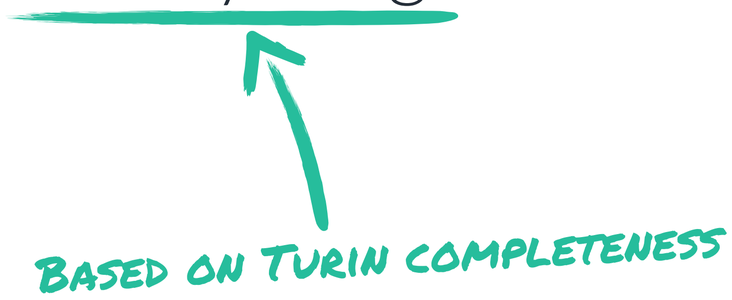
- The EVM is Turin complete
- A Turing complete system can simulate any Turing machine



- The language that is used to program a contract that is executed in the EVM must the Turin complete, too

Ethereum

- Bitcoin is a platform to trade a digital currency
- Ethereum is a platform to do anything



Anything?

Ethereum use cases

- Decentralized Finance (DeFi)
- Non-Fungible Tokens (NFTs)
- Supply Chain Management
- Identity Verification
- Voting Systems
- Decentralized Autonomous Organizations (DAOs)
- Gaming and Virtual Worlds
- Real Estate
- Healthcare
- Legal and Compliance
- Intellectual Property Rights and Royalties

—Ethereum use cases

- Decentralized Finance (DeFi)
- Gaming and Virtual Worlds
- Voting Systems
- Intellectual Property Rights and Royalties
- Decentralized Autonomous Organizations (DAOs)

Ethereum is a so called public ledger

Public Ledger (Definition)

- A public ledger is a **transparent** and **immutable** record of transactions or data accessible to anyone within a network.

- **Transparency:** All transactions are visible to all network participants, fostering transparency.
- **Auditability:** Easy to audit and verify transactions, reducing the risk of fraud.
- **Accessibility:** Anyone with internet access can view the ledger, promoting inclusivity.

OpenElements

Hedera | SWIRLDS LABS

Sky Mavis

skymavis.com

Aktualisieren

SKY MAVIS

Products Team Blog Careers

Blockchain: Gamified

Games with real, player-owned economies will become places where we live, work, and play – true digital nations




What-is-Axie-Infinity-Why-is-t
news.coincu.com/wp-content/uploads/2021/07/What-is-Axie-Infinity-Why-is-this-Play-for-Money.png
Aktualisieren
PatuKasa
Round 1
1
2
3
4
5
6
110
0
30
22
372
456
373
336
420
96
372
0/10
24
Aqua Stock
45
80
4
Kotaro bite
80
20
5
0

OpenElements

Hedera | SWIRLDS LABS

—Your unique Axie



Picture by https://theycb.com

Your unique Axie

HUGE RANGE OF BODY PARTS

CUSTOM FACE EXPRESSION



BASED ON „DNA" OF ANCESTORS

SPECIFIC COLOR

Picture by https://opensea.io

—Your unique Axie

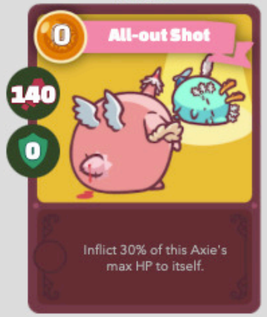


Picture by https://blog.mollywhite.net/axie-hack/

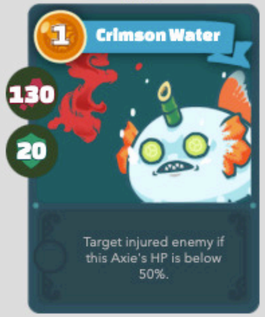
THE LAST ONE
Ranged



POST FIGHT
Ranged



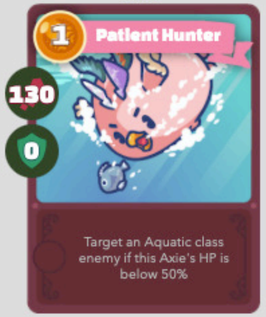
PIRANHA
Melee



DUAL BLADE
Melee



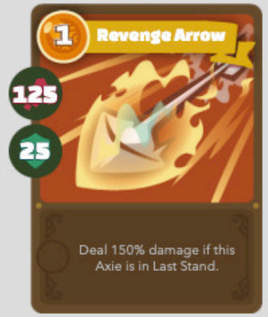
KINGFISHER
Melee



KESTREL
Ranged



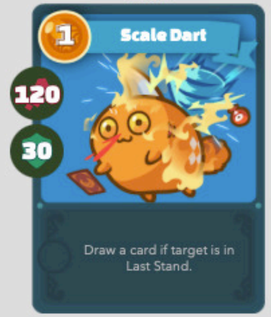
RISKY BEAST
Ranged



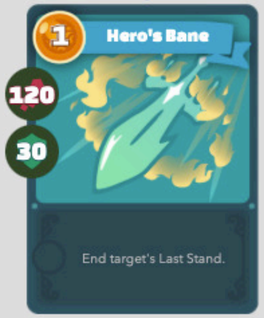
LITTLE BRANCH
Melee



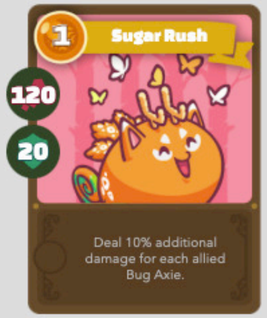
BLUE MOON
Ranged



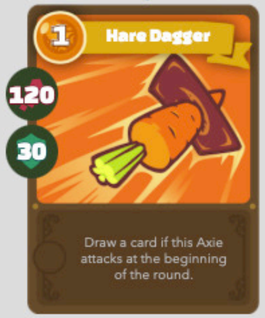
ORANDA
Ranged



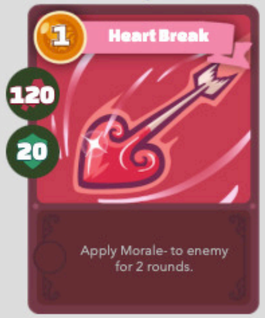
POCKY
Melee



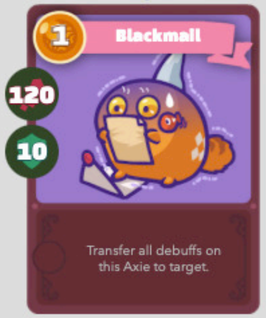
HARE
Ranged



CUPID
Ranged



PIGEON POST
Ranged



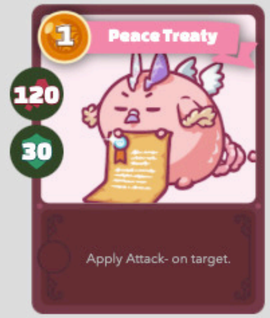
EGGSHELL
Ranged



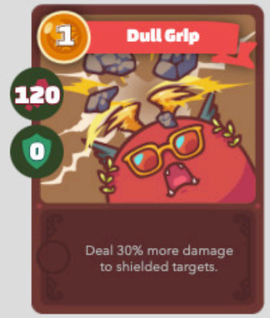
TRUMP
Melee



PEACE MAKER
Melee



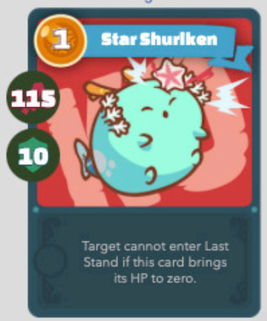
PLIERS
Melee



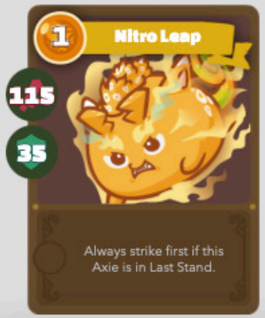
CUTE BUNNY
Melee



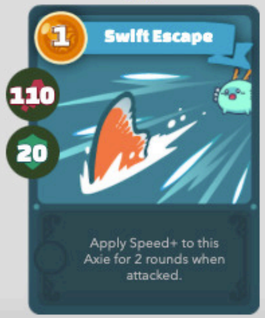
SHOAL STAR
Ranged



JAGUAR
Melee



GOLDFISH
Melee



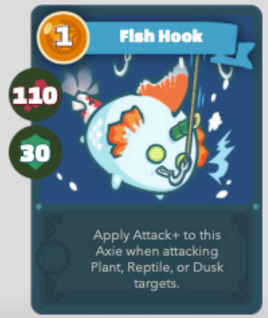
CLAMSHELL
Melee



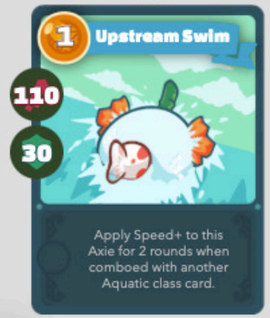
LAM
Melee



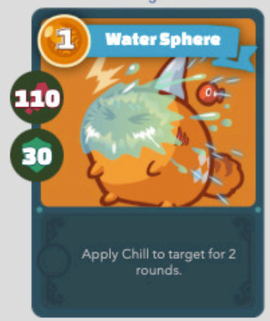
RISKY FISH
Melee



KOI
Melee



RANCHU
Ranged



Picture by https://min.news/zh-hans/game/e74ec5987c336d23d24c72e7427cd373.html

OpenElements

Hedera | SWIRLDS LABS

#3045 ORIGIN
Axie #3045
Breed count: 0



Ξ 1.075 $651.58

#1810 ORIGIN
Axie #1810
Breed count: 0



Ξ 1.125 $681.88

#3002 ORIGIN
Axie #3002
Breed count: 0



Ξ 1.25 $757.65

#4131 ORIGIN
Axie #4131
Breed count: 0



Ξ 1.25 $757.65

#1816 ORIGIN
Axie #1816
Breed count: 0



Ξ 1.2636 $765.89

#3077 ORIGIN
Axie #3077
Breed count: 0



Ξ 1.45 $878.87

#2740 ORIGIN
Axie #2740
Breed count: 0



Ξ 1.5 $909.18

#1060 ORIGIN
Axie #1060
Breed count: 0



Ξ 1.5 $909.18

Picture by https://www.youtube.com/watch?v=u2pu_zfmHLA

#3045 ORIGIN
Axie #3045
Breed count: 0



$651.58

#1810 ORIGIN
Axie #1810
Breed count: 0



$681.88

#3002 ORIGIN
Axie #3002
Breed count: 0



$757.65

#4131 ORIGIN
Axie #4131
Breed count: 0



$757.65

#1816 ORIGIN
Axie #1816
Breed count: 0



$765.89

#3077 ORIGIN
Axie #3077
Breed count: 0



$878.87

#2740 ORIGIN
Axie #2740
Breed count: 0



$909.18

#1060 ORIGIN
Axie #1060
Breed count: 0



$909.18

Picture by https://www.youtube.com/watch?v=u2pu_zfmHLA

Sustainability

Sustainability of Bitcoin

The power of the bitcoin network could be used to power all tea kettles of the UK to boil water



—Sustainability of Bitcoin

The power of the bitcoin network could be used to power all tea kettles of the UK to boil water

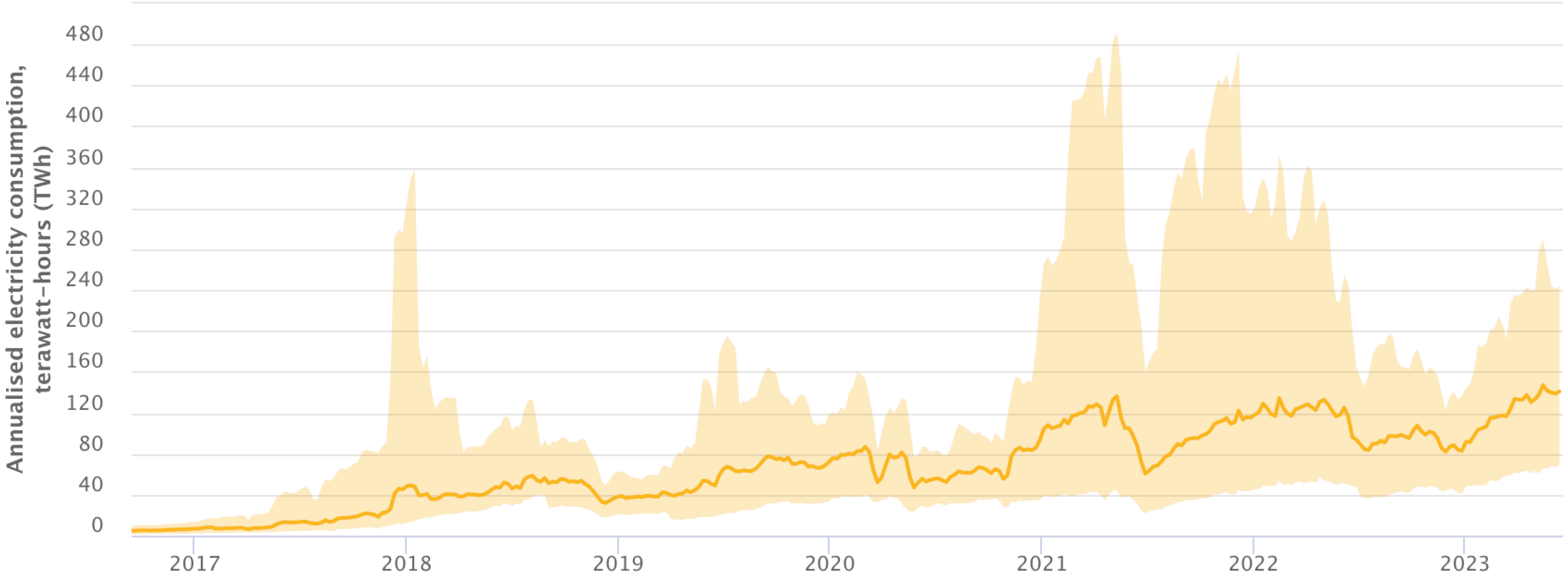


...for 30 years

OpenElements

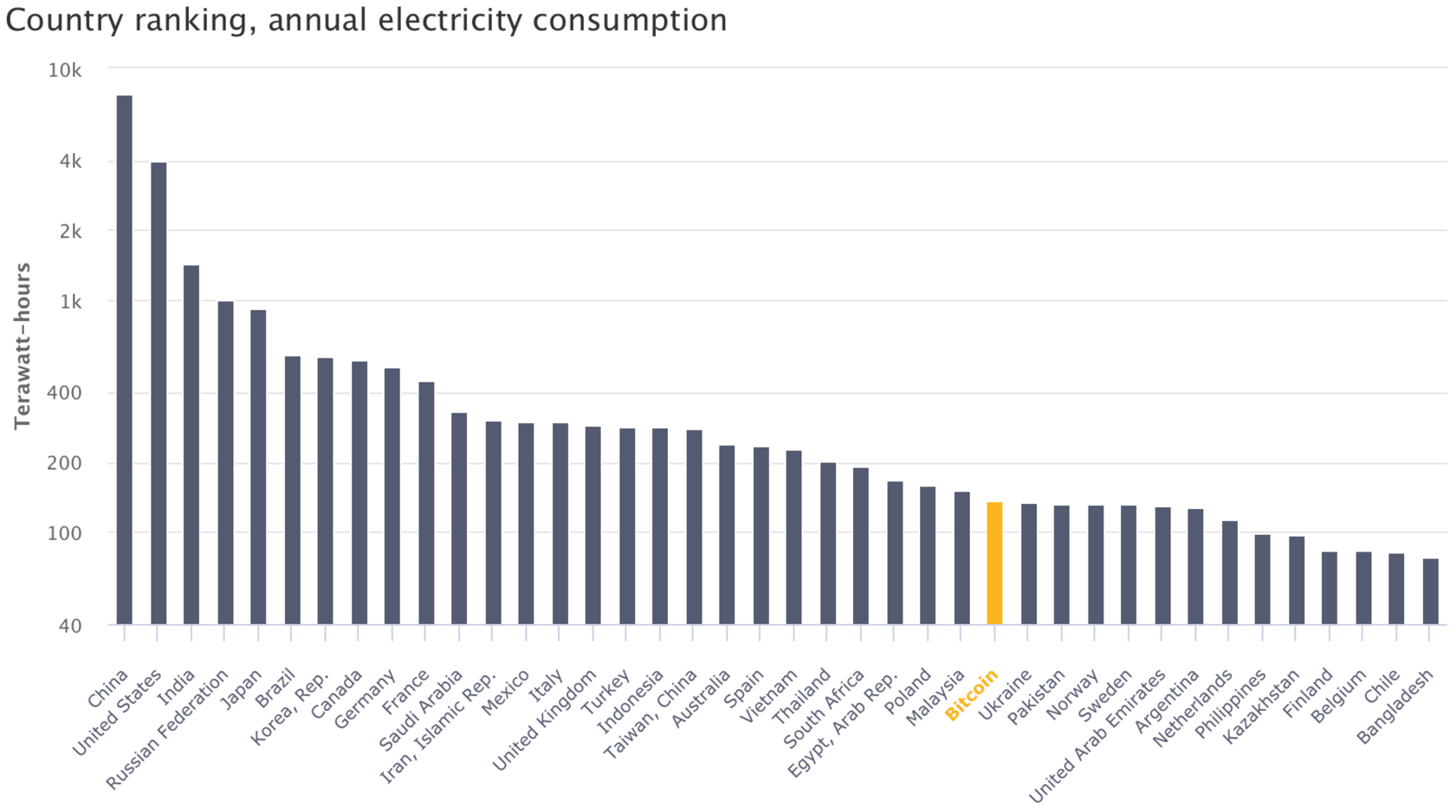
Hedera | SWIRLDS LABS

Sustainability of Bitcoin



https://ccaf.io/cbnsi/cbeci

—Sustainability of Bitcoin

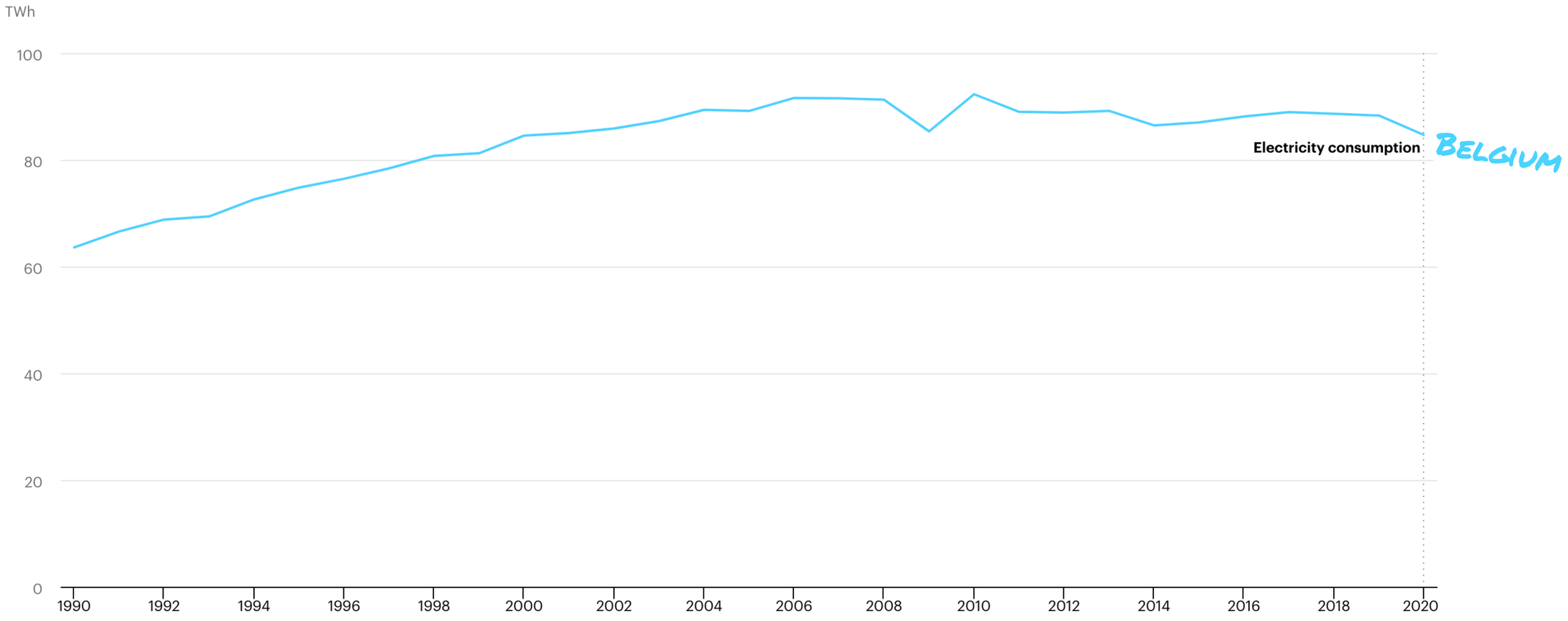


https://ccaf.io/cbnsi/cbeci

OpenElements

Hedera | SWIRLDS LABS

Sustainability of Ethereum



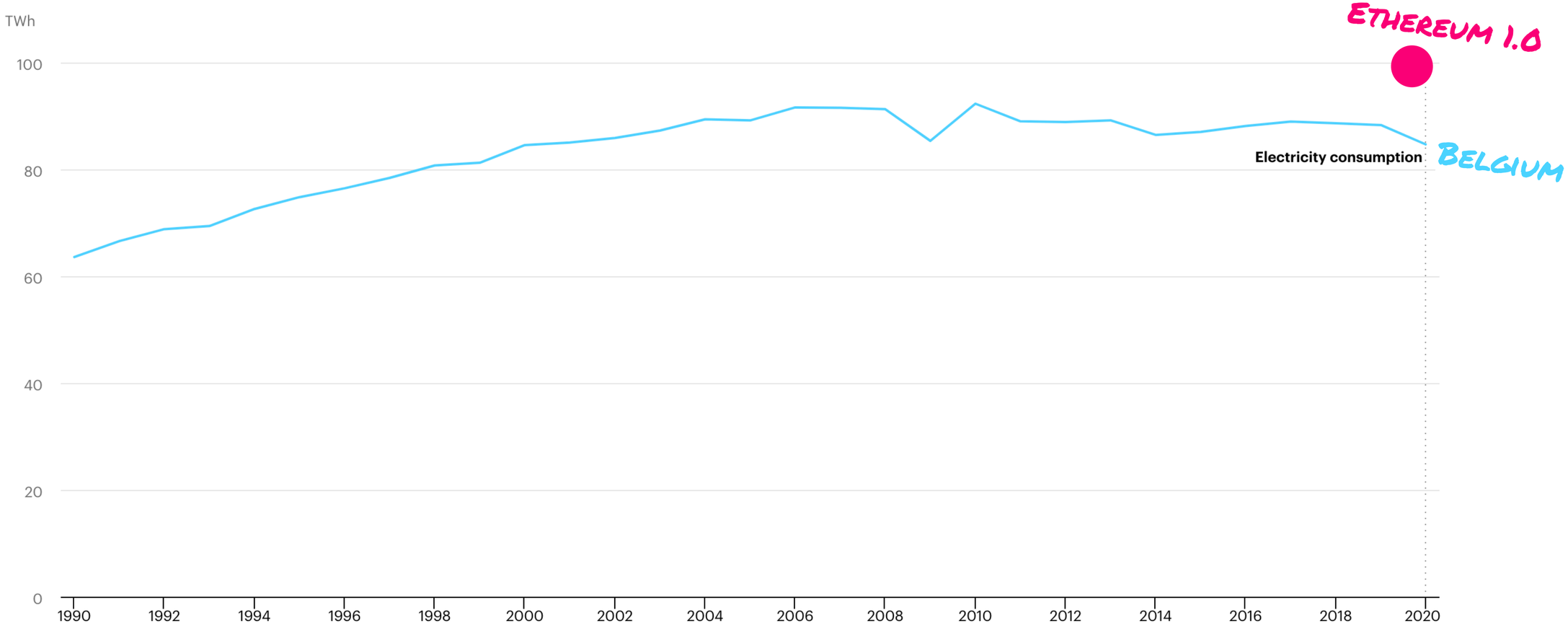
https://www.iea.org/countries/belgium

https://www.sciencefocus.com/future-technology/can-nfts-solve-their-massive-carbon-footprint-problem/

OpenElements

Hedera | SWIRLDS LABS

Sustainability of Ethereum



https://www.iea.org/countries/belgium

https://www.sciencefocus.com/future-technology/can-nfts-solve-their-massive-carbon-footprint-problem/

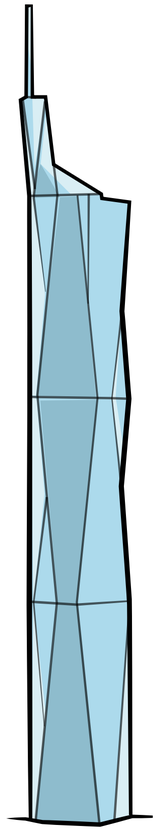
Sustainability of Ethereum

- **Proof of Work:** yearly electricity consumption of **22,000,000 TWh** and **10,000,000 t CO_2** emission

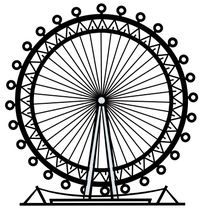


- **Proof of Stake:** yearly electricity consumption of **2000 MWh** and **650 t CO_2** emission

—Sustainability of Ethereum

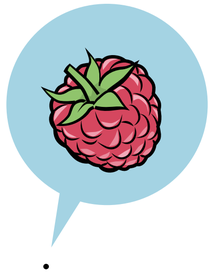


Bitcoin



Ethereum 1.0

From Proof-of-Work to Proof-of-Stake



Ethereum 2.0

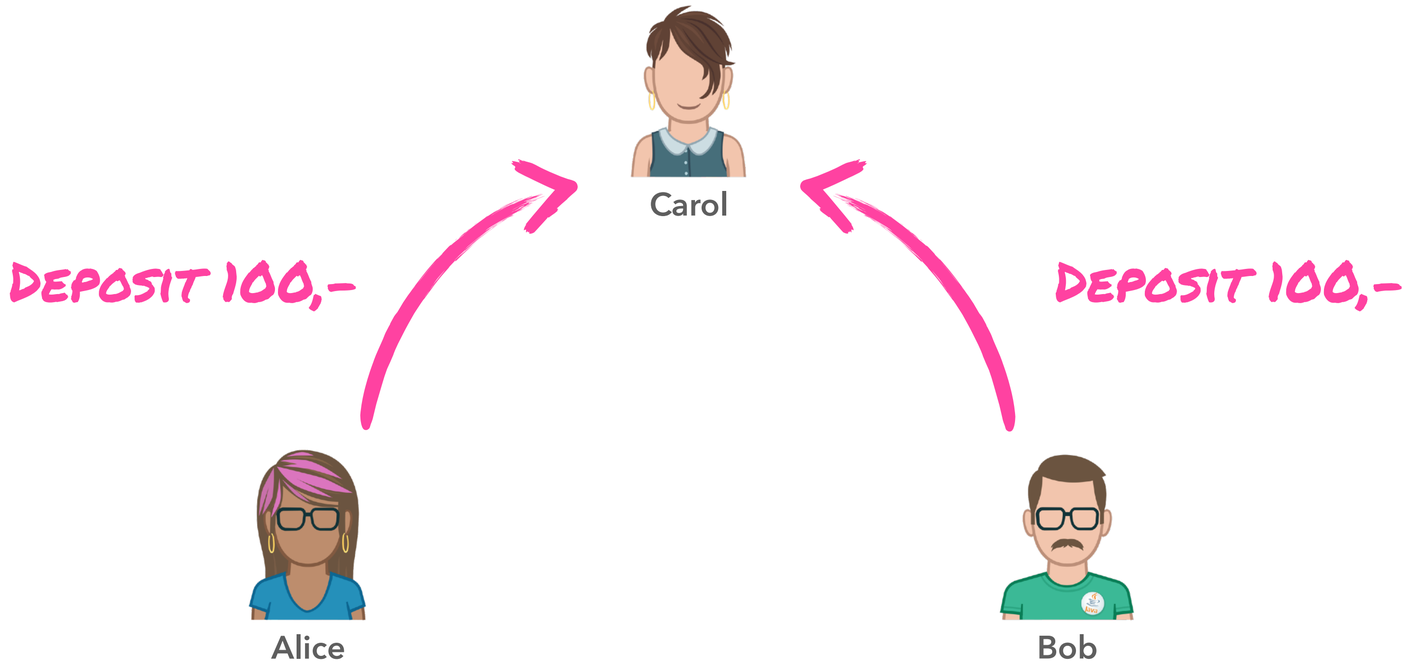
—Sustainability of networks

- **PayPal:** yearly electricity consumption of 0.28 TWh
- **Netflix:** yearly electricity consumption of 0.45 TWh
- **YouTube:** yearly electricity consumption of 12 TWh

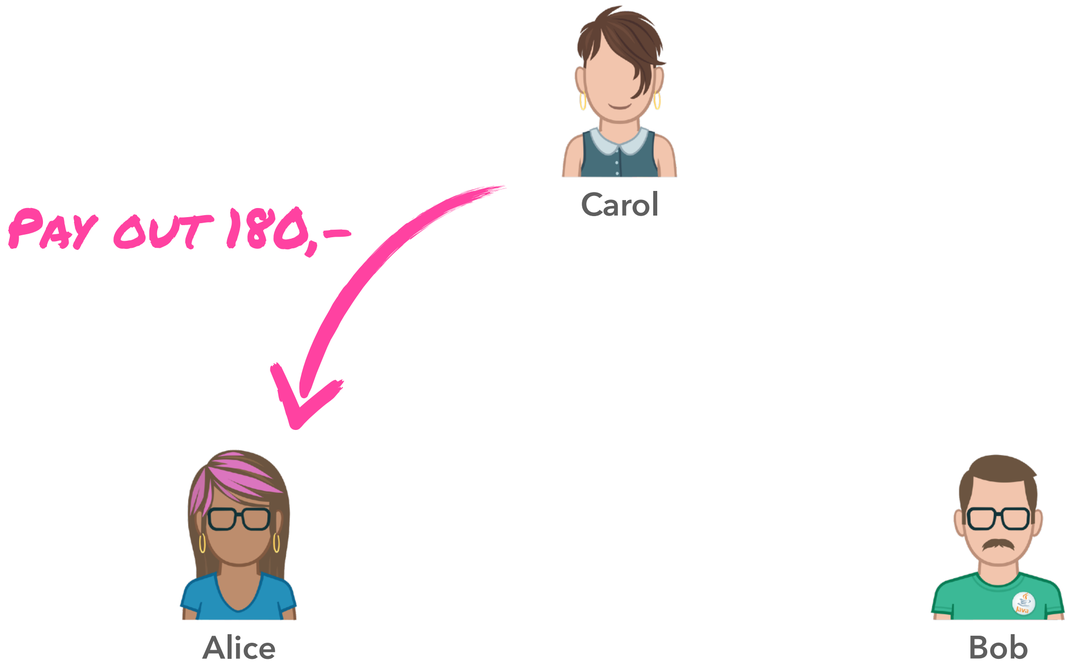
- **Bitcoin:** yearly electricity consumption of 131 TWh
- **Ethereum:** yearly electricity consumption of 0.0026 TWh

Smart Contracts and the EVM

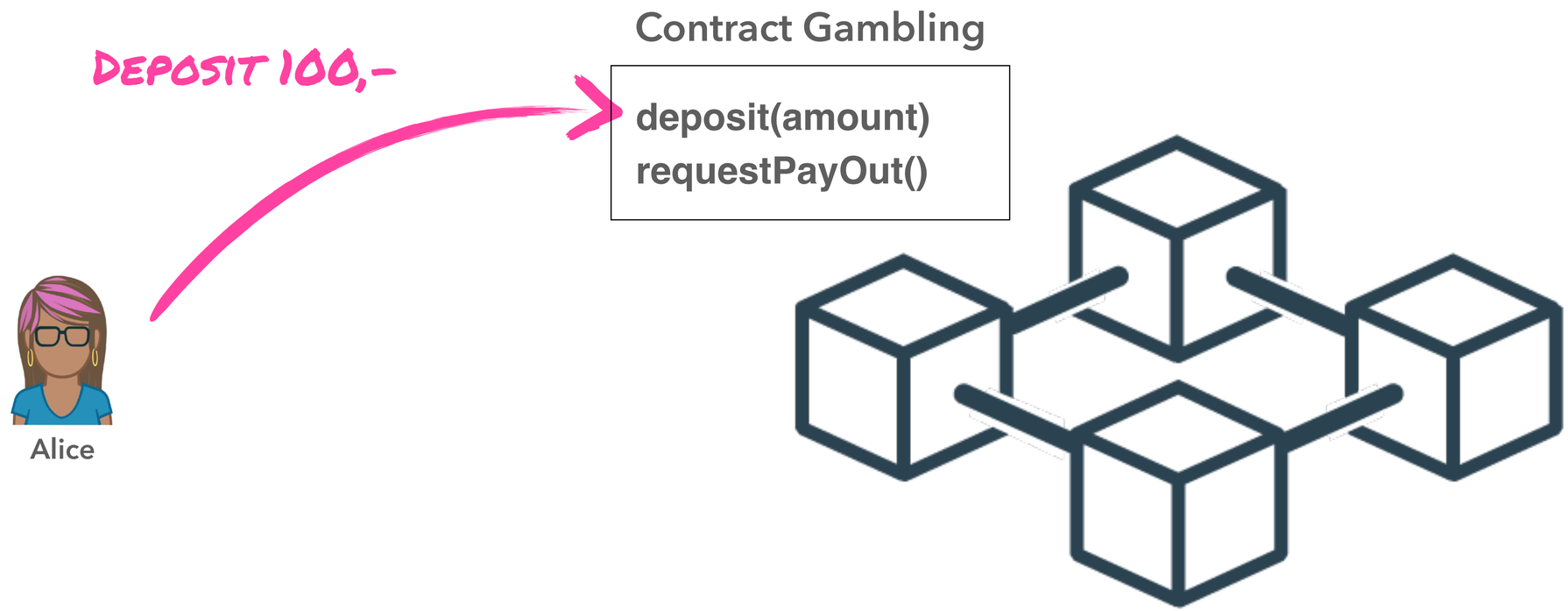
Smart Contract: Gambling



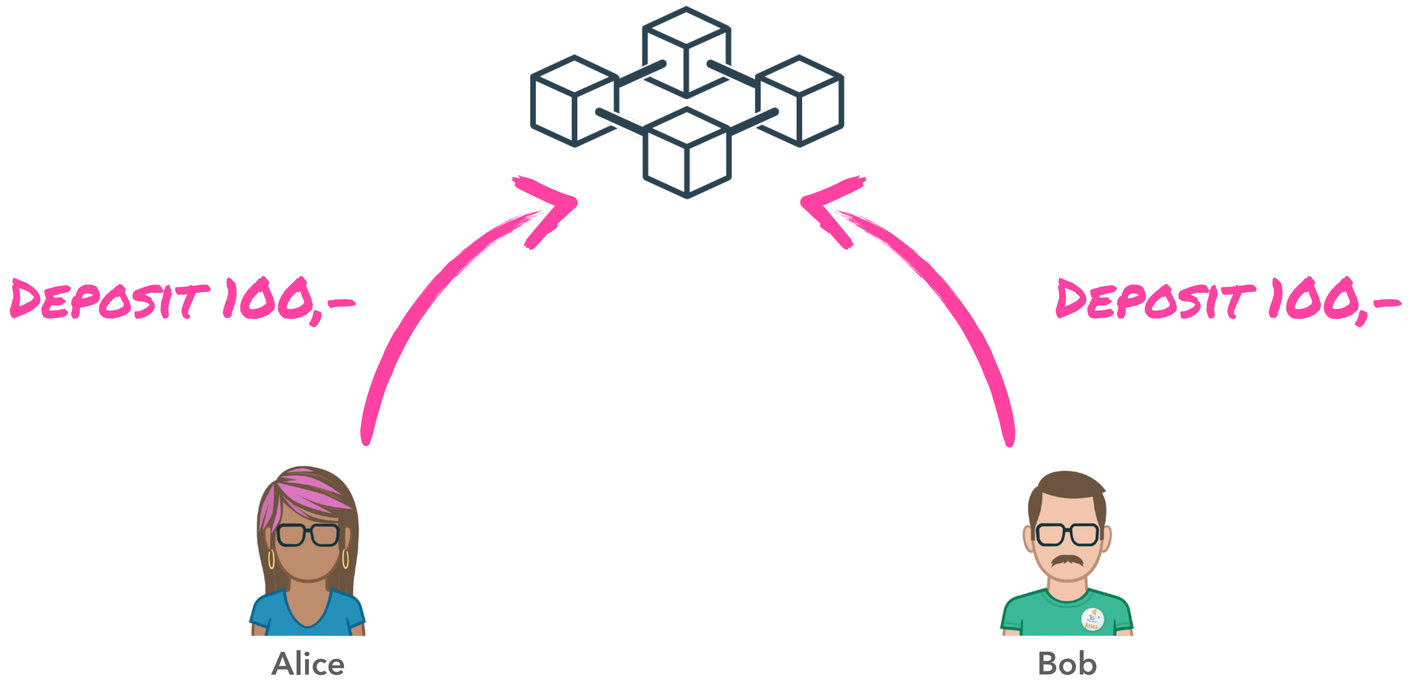
—Smart Contract: Gambling



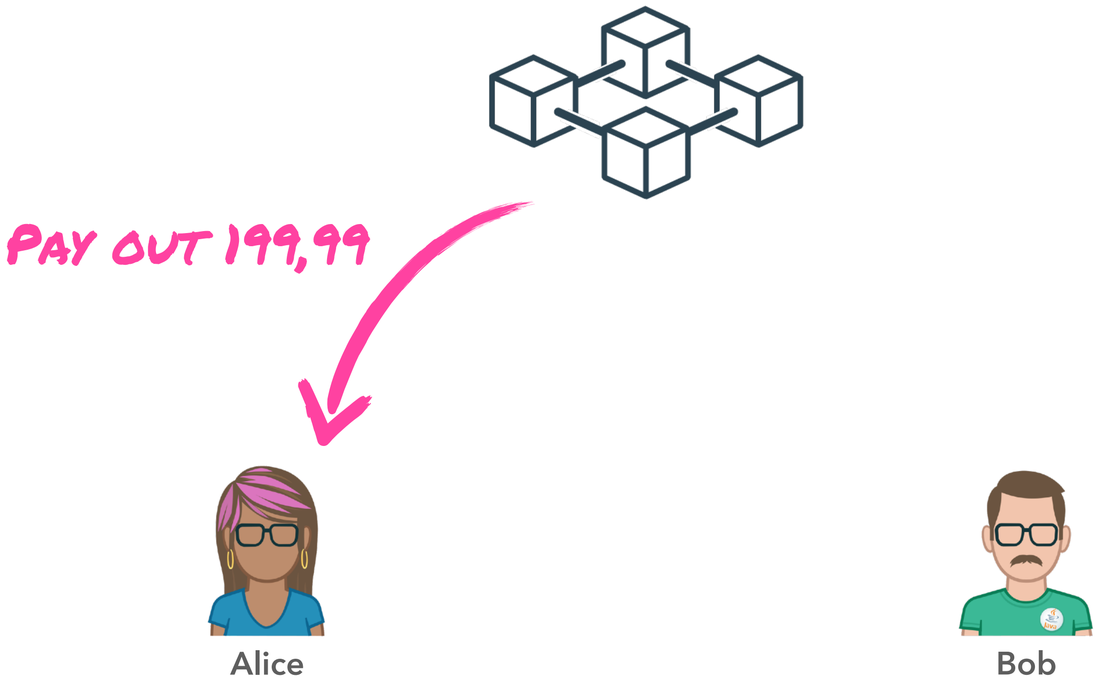
—Smart Contract: Gambling



Smart Contract: Gambling



—Smart Contract: Gambling

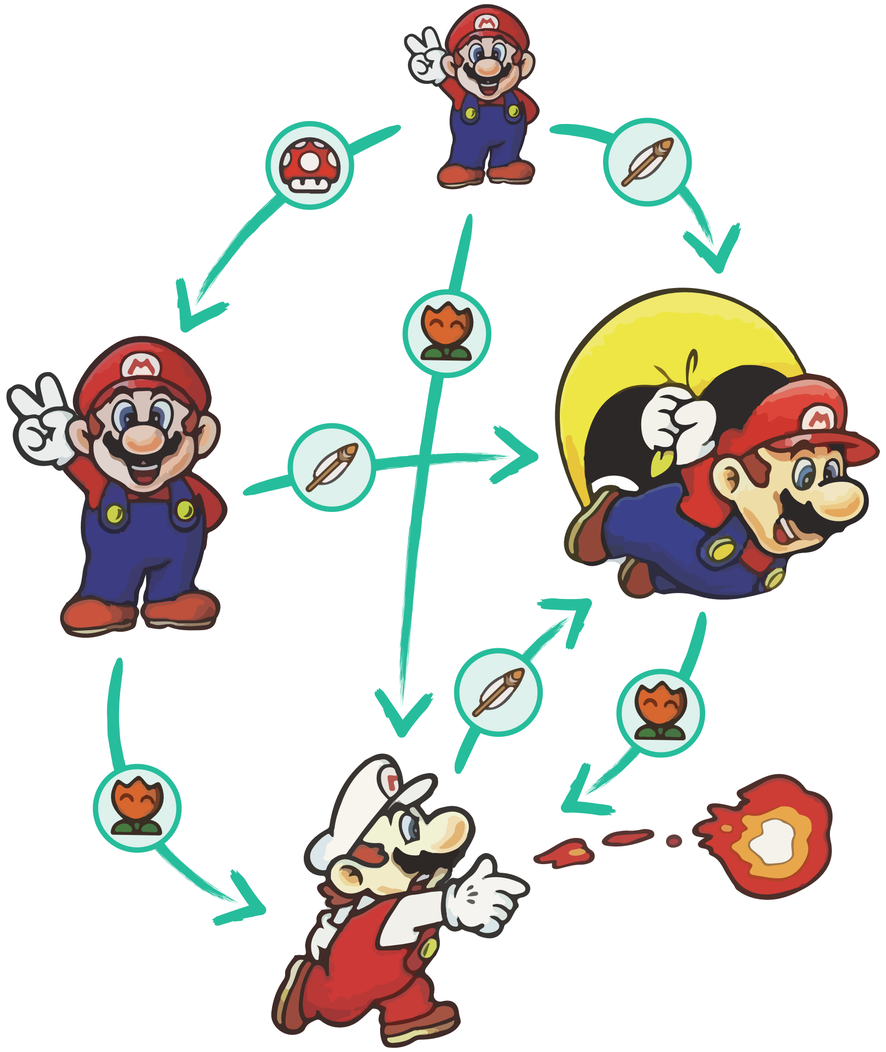


—Ethereum Virtual Machine

- Ethereum is a distributed **state machine**
- Ethereum's state is a large data structure which holds not only all accounts and balances, but a **machine state**
- The specific rules of changing state from block to block are defined by the **EVM**

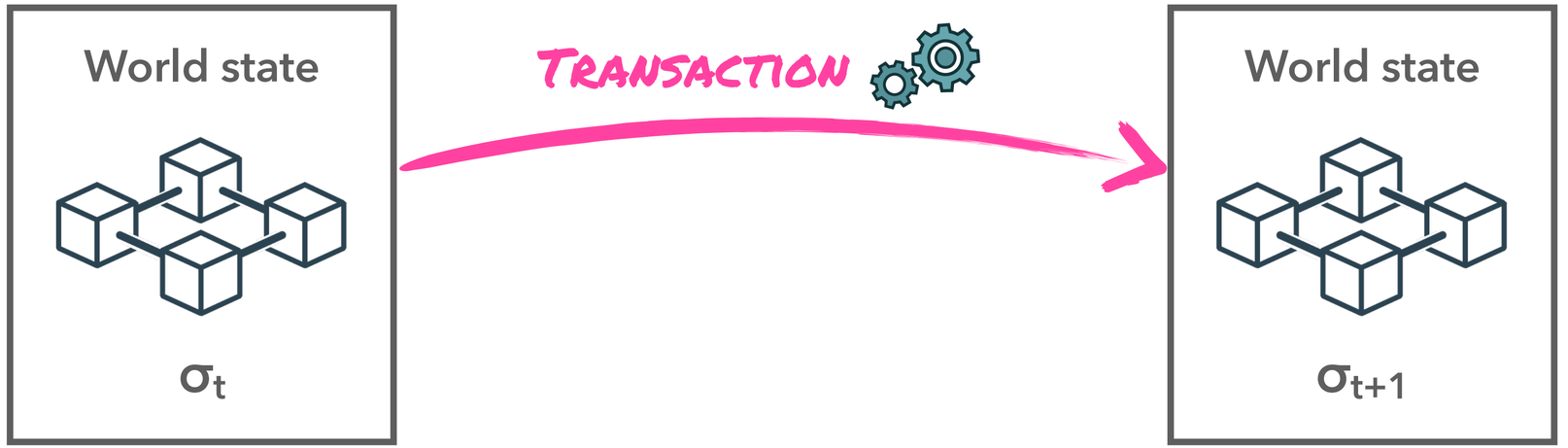
State Machine

- Model used to design computer programs and sequential logic circuits
- **States:** A state machine consists of a finite number of states. At any given time, the machine is in one of these states.
- **Transitions:** Changes in state, known as transitions, occur in response to external inputs or conditions.
- **Initial State:** There is usually a designated starting state where the machine begins its operation.

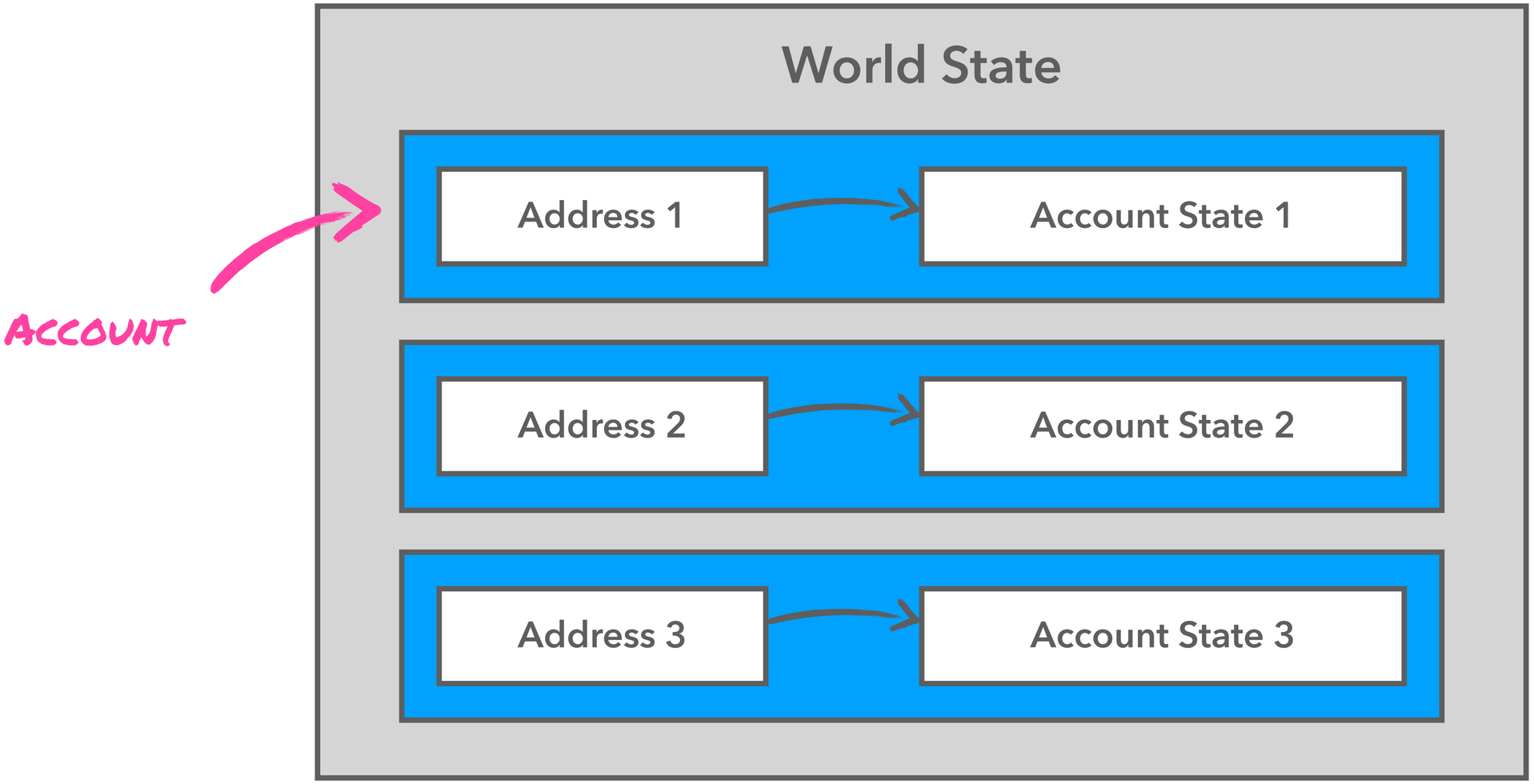


Picture by https://www.mariowiki.com/Super_Mario_World

—Ethereum State Machine



Ethereum State Machine



Executing contracts

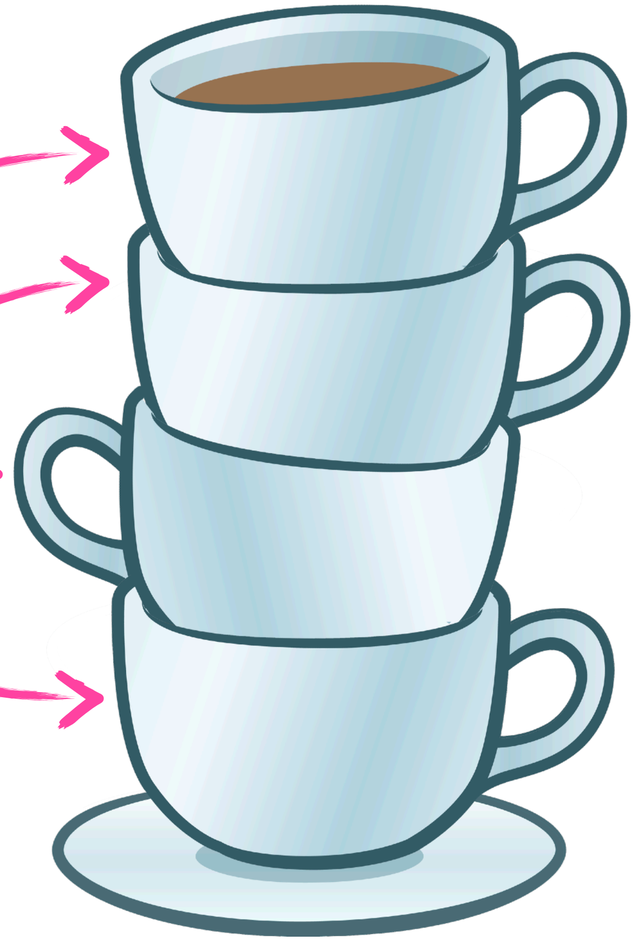
- **Program Counter (PC)** defines what operation (OpCode) should be handled next.
- Normally incremented by 1 (1 Byte)
- Some special operations like `JUMP` change the Program Counter in a more specific way (in conjunction with the stack)

Executing contracts

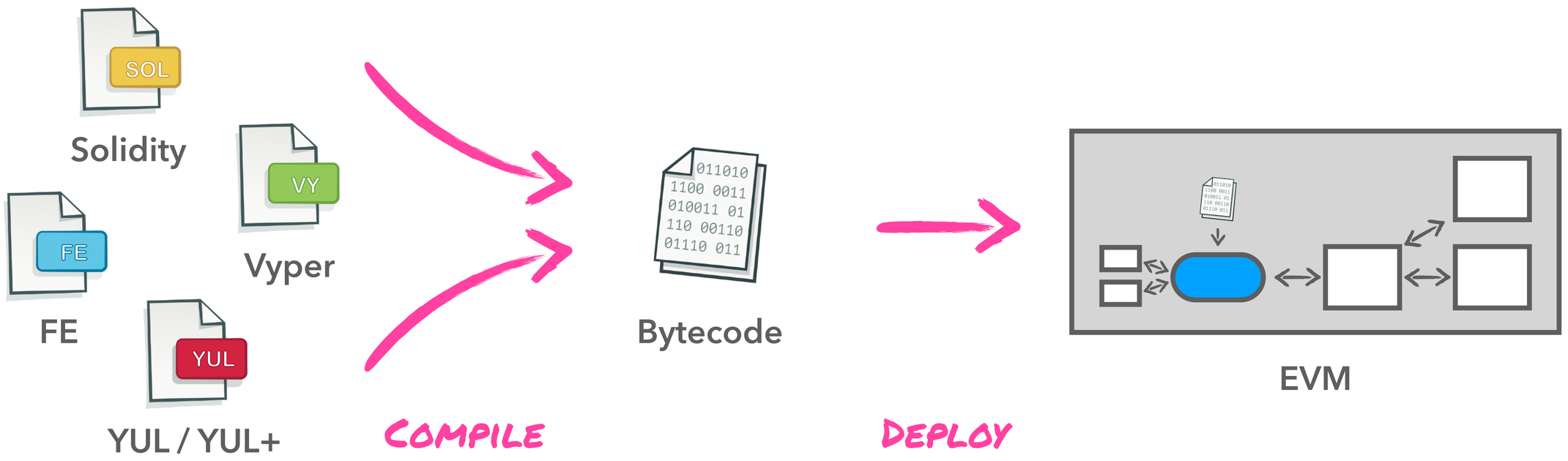
- **Stack** is a list of max 1024 elements
- An element defines a step in call hierarchy of an application
- The top element always defines the current state of the current call context
- Maybe you know stack traces from exceptions and errors

—Stack Trace sample (Java)

```
Exception in thread "main" java.lang.NullPointerException
        at com.example.myproject.Book.getTitle(Book.java:16)
        at com.example.myproject.Author.getBookTitles(Author.java:25)
        at com.example.myproject.Controller.showBooks(Controller.java:58)
        at com.example.myproject.Bootstrap.main(Bootstrap.java:14)
```



—Ethereum Virtual Machine



—Executing contracts

- Smart contract is executed by performing low-level instructions, so called OpCodes Operation Codes (**OpCodes**)
- EVM is a stack-based virtual machine, where OpCodes interact with a data stack and manipulate values

EVM OpCodes

Stack	Name	Gas	Initial Stack	Resulting Stack	Notes
00	STOP	0			halt execution
01	ADD	3	a, b	a + b	(u)int256 addition modulo 2**256
02	MUL	5	a, b	a * b	(u)int256 multiplication modulo 2**256
03	SUB	3	a, b	a - b	(u)int256 addition modulo 2**256
04	DIV	5	a, b	a // b	uint256 division
05	SDIV	5	a, b	a // b	int256 division
06	MOD	5	a, b	a % b	uint256 modulus
07	SMOD	5	a, b	a % b	int256 modulus

EVM OpCodes

Stack	Name	Gas	Initial Stack	Resulting Stack	Notes
00	STOP	0			halt execution
01	ADD	3	a, b	a + b	(u)int256 addition modulo 2**256
02	MUL	5	a, b	a * b	(u)int256 multiplication modulo 2**256
03	SUB	3	a, b	a - b	(u)int256 addition modulo 2**256
04	DIV	5	a, b	a // b	uint256 division
05	SDIV	5	a, b	a // b	int256 division
06	MOD	5	a, b	a % b	uint256 modulus
07	SMOD	5	a, b	a % b	int256 modulus

Gas definition

- When creating a transaction a gas value needs to be defined
- The value defines the maximum of gas that the transaction can cost
- Transaction will be aborted if the cost is too high



```
transaction failed pre-check with
the status `INSUFFICIENT_GAS`
```

Hands on lesson

- Why is gas an important concept?
- What is a benefit of a VM?
- What could be differences between the EVM and the JVM (Java Virtual Machine)

Solidity
Introduction

Solidity

- The Solidity language documentation can be found at

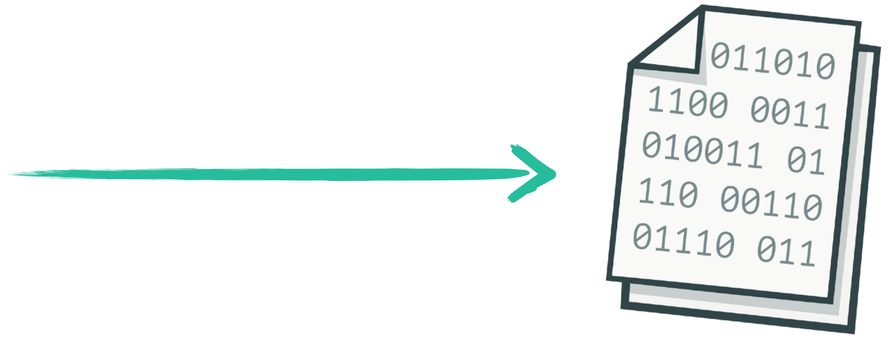
https://docs.soliditylang.org/

—Solidity

- The Solidity language documentation can be found at

```
pragma solidity ^0.8.20;

contract HelloWorld {
    string public greet = "Hello World!";
}
```



compiled contract

OpenElements

Hedera | SWIRLDS LABS

—Solidity

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

—Solidity

Version Pragma

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

OpenElements

Hedera | SWIRLDS LABS

—Solidity

CONTRACT



```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

—Solidity

State Variables

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Solidity

Events



```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

—Solidity

CONSTRUCTOR

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

—Solidity

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```



OpenElements

Hedera | SWIRLDS LABS

—Solidity

```
pragma solidity >= 0.7.0;

contract Coin {

    address public minter;
    mapping (address => uint) public balances;

    event Sent(address from, address to, uint amount);

    constructor() {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        require(msg.sender == minter);
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        require(amount <= balances[msg.sender]);
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Compile Solidity

- To execute the smart contract we need to compile it
- The compilation is normally stored in a binary BIN file



Smart Contract

solc
Solidity Compiler

compiled contract

Compile Solidity

- The Solidity compiler solc can easy be installed locally

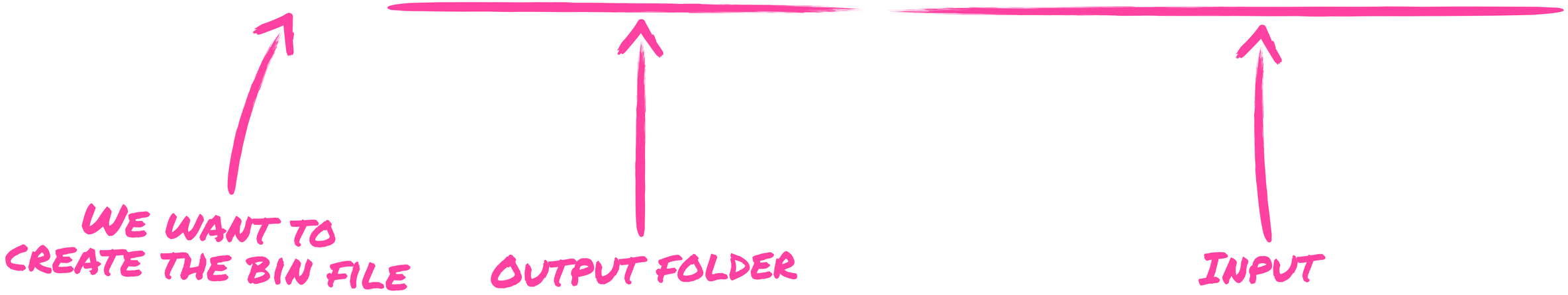
`https://docs.soliditylang.org/`

- The compiler provides different ways how it can be installed locally (brew, npm, …)

Compile Solidity

- We can easily compile our smart contract by using solc from the commandline:

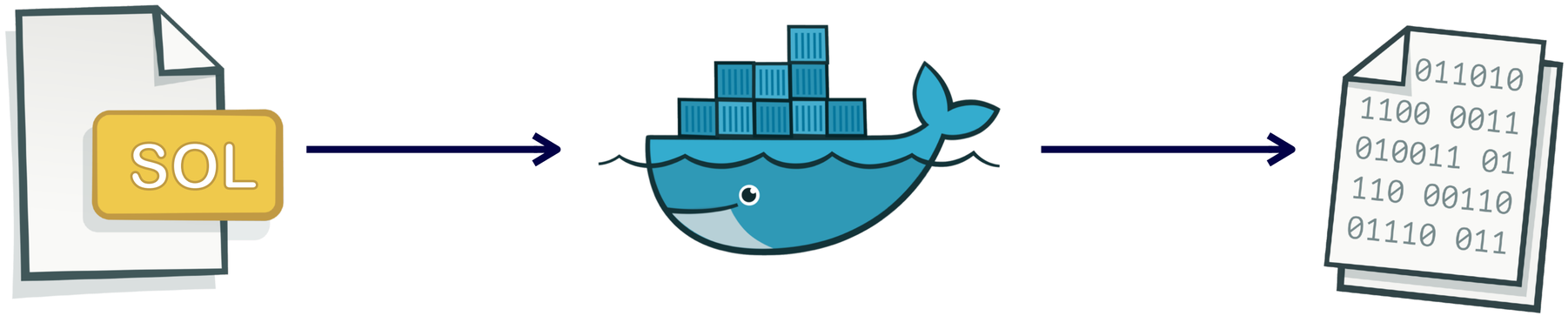
```
solc --bin -o build/contracts contracts/hello_world.sol
```



Compile Solidity

- Instead of installing the compiler locally you can use it wrapped in a docker container

```
docker run -v $(pwd)/contracts:/contracts ethereum/solc:stable -o /contracts/output --bin
```

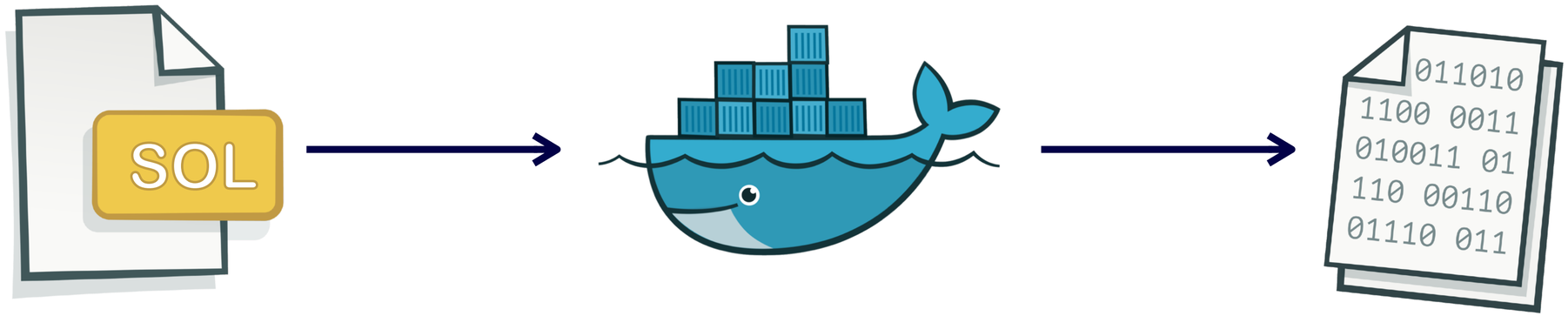


Smart Contract → solc container → compiled contract

Compile Solidity

- Instead of installing the compiler locally you can use it wrapped in a docker container

```
docker run -v $(pwd)/contracts:/contracts ethereum/solc:stable -o /contracts/output --bin
```



Smart Contract → solc container → compiled contract

OpenElements

Hedera | SWIRLDS LABS

Hands on lesson

- Create a "hello world" smart contract
- What editor can be used?
- Compile the contract without errors

Solidity Part 2

—Deploy a Smart Contract

1. Write the contract in solidity




```
pragma solidity >= 0.7.0;

contract myContract {

    address owner;

    constructor() {
        owner = msg.sender;
    }

    function sayHi() public returns(string memory) {
        Return "Hi";
    }
}
```

—Deploy a Smart Contract

1. Write the contract in solidity
2. Compile the contract




```
pragma solidity >= 0.7.0;

contract myContract {

    address owner;

    constructor() {
        owner = msg.sender;
    }

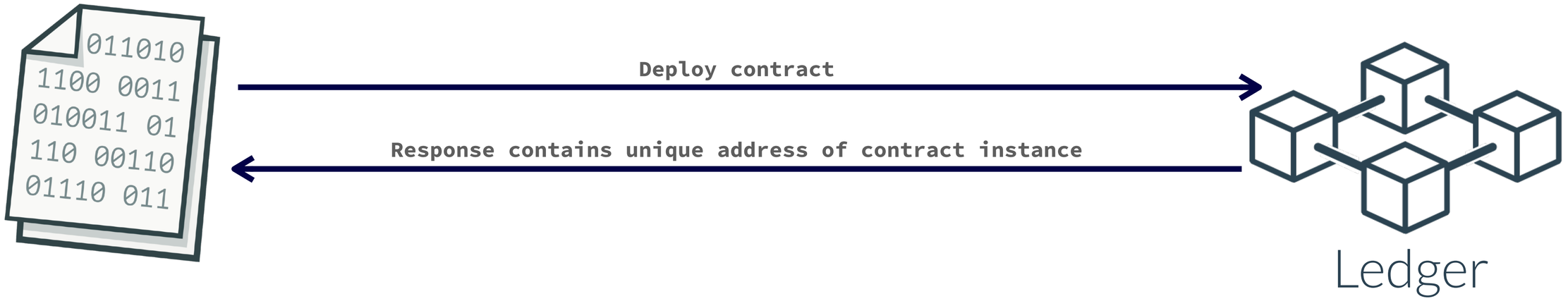
    function sayHi() public returns(string memory) {
        Return "Hi";
    }
}
```

```
solc --bin -o build/contracts contracts/hello_world.sol
```



Deploy a Smart Contract

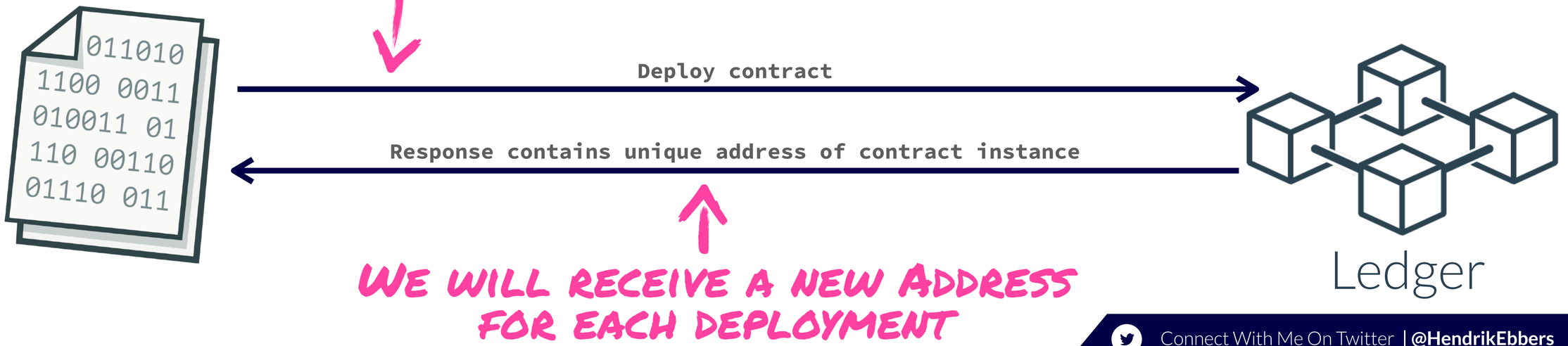
1. Write the contract in solidity
2. Compile the contract
3. Deploy the contract



Deploy a Smart Contract

1. Write the contract in solidity
2. Compile the contract
3. Deploy the contract

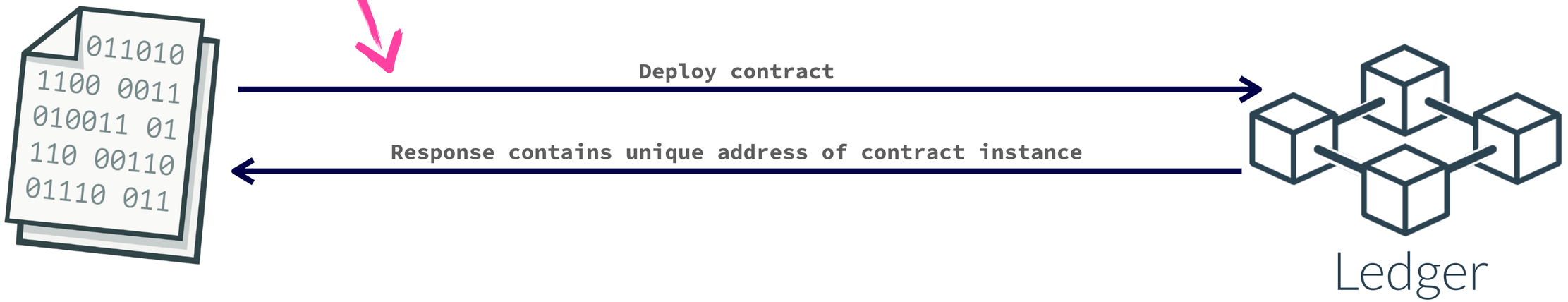
A CONTRACT CAN BE DEPLOYED MULTIPLE TIMES



—Deploy a Smart Contract

1. Write the contract in solidity
2. Compile the contract
3. Deploy the contract

It is like "each deployment instantiates a new contract instance"

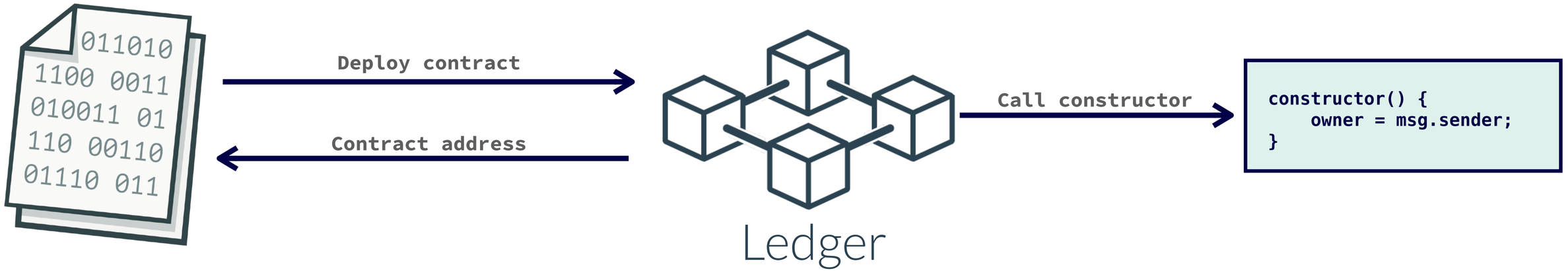


OpenElements

Hedera | SWIRLDS LABS

—Deploy a Smart Contract

1. Write the contract in solidity
2. Compile the contract
3. Deploy the contract (instance)



Smart Contract lifecycle

- When a contract is deployed to a ledger an instance is created
- That instance is the only instance that will ever exist of the deployed contract
- The instance has a unique address (like a user) in the ledger
- The instance "holds" the state of the contract

Transactions

Common properties of Transaction for Hedera

Method	Type
`getTransactionID()`	TransactionID
`getTransactionValidDuration()`	Duration
`getTransactionMemo()`	String
`getNodeAccountId()`	AccountID
`getMaxTransactionFee()`	Hbar
`getTransactionHash()`	byte[ ]
`getTransactionHashPerNode()`	Map<AccountId, byte [ ]>
`getSignatures()`	Map<AccountId, Map<PublicKey, byte [ ]>>

—Solidity

- Every contract has a name

```
contract MyContract {

}
```

Solidity

- A contract can have functions
- A function must have a unique name
- A function must? Have a visibility def
  - Public / private (more will come later)

```
contract MyContract {

function hello() public { }

}
```

—Solidity

- Functions can have parameters
- A parameter is defined by a type and a name

```
contract MyContract {

function hello(int amount) public { }

function calculate(int a, int b) public { }

}
```

—Solidity

- Functions can have a return type
- A return type is defined by a type

```
contract MyContract {

function getCount() public returns (int)
}
```

Solidity

- Solidity support several types
- Let's start by using `int` and `bool`
- Info: `int` can hold a 256 bits int value

```
contract MyContract {

function getCount() public returns (int)

function isActive() public returns (bool)

}
```

—Solidity

- State variables

State Variables

State variables are variables whose values are permanently stored in contract storage.

open in Remix

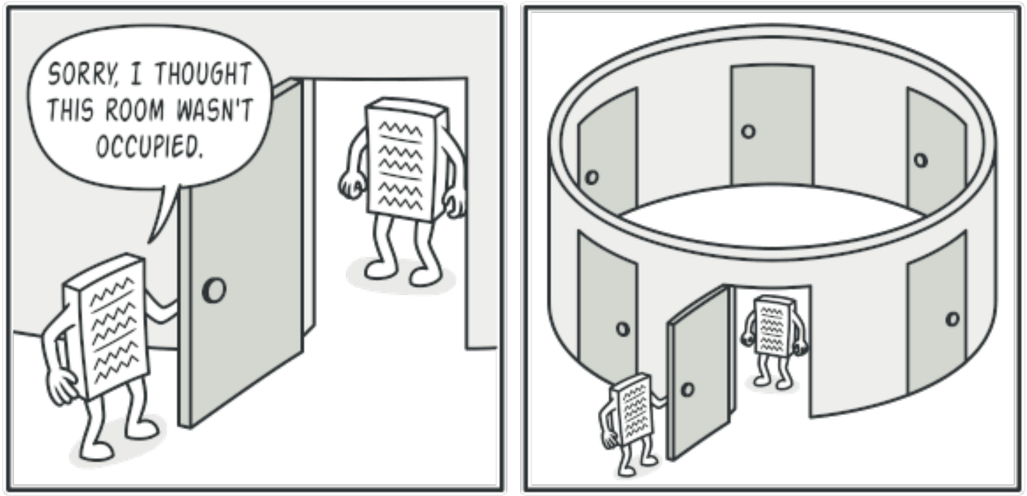
```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.0 <0.9.0;

contract SimpleStorage {
    uint storedData; // State variable
    // ...
}
```

See the Types section for valid state variable types and Visibility and Getters for possible choices for visibility.

—Solidity

- Constructor and the lifecycle of a contract and its state
- You can thing about a singleton



—Value Types

- Boolean
- Integers
  (int, int8, int16, …, int256, uint, uint8,
  uint16, …, uint256)
- Fixed Point Numbers (👷)
- Address
- Byte Arrays (fixed and dynamically-sized)
- Enums

Reference Types

- Array
- Map
- Struct

Control Structures

- if, else
- while, do
- for
- break, continue
- return

Error Handling

- state-reverting
- try-catch
- require
- revert

—ModifiersModifiers

Modifiers

- `pure` for functions: Disallows modification or access of state.
- `view` for functions: Disallows modification of state.
- `payable` for functions: Allows them to receive Ether together with a call.
- `constant` for state variables: Disallows assignment (except initialisation), does not occupy storage slot.
- `immutable` for state variables: Allows assignment at construction time and is constant when deployed. Is stored in code.
- `anonymous` for events: Does not store event signature as topic.
- `indexed` for event parameters: Stores the parameter as topic.
- `virtual` for functions and modifiers: Allows the function's or modifier's behavior to be changed in derived contracts.
- `override` : States that this function, modifier or public state variable changes the behavior of a function or modifier in a base contract.

—Working with Contracts

- Delegate: `https://solidity-by-example.org/delegatecall/`
- Calling contracts: `https://solidity-by-example.org/calling-contract/`
- Inheritance: `https://docs.soliditylang.org/en/v0.8.23/contracts.html#inheritance`
- Using for (structs) : `https://docs.soliditylang.org/en/v0.8.23/contracts.html#using-for`

—Working with Libraries

- Libraries: `https://docs.soliditylang.org/en/v0.8.23/contracts.html#libraries`
- Simple Sample: `https://docs.soliditylang.org/en/v0.8.23/solidity-by-example.html#modular-contracts`
- Import: `https://docs.soliditylang.org/en/v0.8.23/layout-of-source-files.html#importing-other-source-files`
- Import in Remix: `https://remix-ide.readthedocs.io/en/latest/import.html#import-from-a-github-url`